```
  public function getAuthPassword();
  public function getRememberToken();
  public function setRememberToken($value);
  public function getRememberTokenName();
}
```

This interface is straightforward. The **`getAuthIdentifierName`** function should return the name of the user's **`primary key`** field, and the **`getAuthIdentifier`** method should return the user's **`primary key.`** When utilizing a MySQL backend, this is most likely the user record's auto-incrementing primary key. The **`getAuthPassword`** function should return the hashed password of the user.

This interface enables the authentication system to interact with any **user** class, independent of the ORM or storage abstraction layer used. Laravel provides an **`App\Models\User`** class in the **`app/Models`** directory by default, which implements this interface.

Implementing Authorization Policies and Gates

While incorporating security with authentication and authorization, a key aspect is to keep track of the users currently allowed into the system. The users who are authenticated and authorized should also be kept in check and have a policy defined to auto-logout and keep track of the session data.

You may use the Auth facade's logout function to manually log users out of your application. This removes the authentication information from the user's session, preventing future requests from being authenticated.

It is suggested that, in addition to invoking the logout function, you invalidate the user's session and renew their CSRF token. After logging the user out, you should normally redirect the user to the application's root:

```
use Illuminate\Http\Request;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
/**
  * Log the user out of the application.
  */
public function logout(Request $request): RedirectResponse
{
```